# Enterprise Product Requirements Feature-Gap Analysis

## Assessment status

| Field | Value |
|---|---|
| Status | Enterprise-horizon rebaseline complete |
| Assessed | 2026-08-30 (America/Chicago) |
| Target horizon | Enterprise-level product approximately one year from assessment |
| Requirements source | `docs/Architecture/Product_Requirements.md` |
| Implementation assessed | `Z:\mvp20-split` working tree |
| Scope | Repository-visible product, science, capability-node, enterprise, UI, audit, and agent-orchestration coverage |
| Overall estimate | Approximately **18% implemented / 82% feature gap** against the expanded enterprise target |
| Scientific readiness | Pre-validation: 29/29 registered modules are `UNVALIDATED`; 0 are ranking-eligible |
| Confidence | High for repository-visible behavior; private deployments, omitted data, other branches, and external lab work were not assessed |

## Executive conclusion

Amphora is a substantial **scientific trust, diagnostics, and execution kernel**, not yet an enterprise materials R&D product. Its strongest assets are strict contracts, canonical identity and hashes, lineage, replay, signed receipts, source-rights controls, capability routing, authority ceilings, fail-closed optimizer authorization, retained rejects/duplicates, experimental evidence bindings, packaging primitives, and a broad initial coatings-oriented module set.

The requirements now describe a much larger product: a multi-tenant system of record, scientist workbench, laboratory integration platform, extensible capability-node ecosystem, enterprise identity and integration surface, reliable service platform, customer portal, cross-domain audit service, and agent-orchestrated support environment. Most of those product surfaces do not exist. The current implementation is predominantly Rust libraries, CLI/TUI workflows, immutable files, fixtures, evidence packages, and diagnostic campaigns. No repository-visible browser application, service API, enterprise IAM, multi-tenant transactional store, public integration platform, general node SDK/lifecycle service, or active product agent service was found.

The earlier assessment estimated roughly 30% implemented against a narrower MVP denominator. The enterprise rebaseline is approximately **18% implemented / 82% gap**. This reflects expanded scope, not regression.

For one year, the credible goal is an **enterprise coatings R&D pilot**: secure tenant foundations, one complete coatings/naval workflow, a minimum viable capability-node platform, role-based web UI, cross-domain auditability, lab-return closure, and bounded agent assistance. Attempting every deployment profile, enterprise integration, commercial system, material domain, scientifically validated ranking model, and high-autonomy agent behavior in that year would be high risk.

## Scoring method

- `Implemented` — supported end-to-end workflow usable by its intended role.
- `Foundation` — real production-relevant kernel behavior without a complete product workflow.
- `Partial` — meaningful implementation with major lifecycle, persistence, authority, or UI gaps.
- `Skeleton` — schema, fixture, diagnostic, audit-only path, or documentation.
- `Missing` — no repository-visible implementation.

| Requirement group | Weight | Coverage | Weighted contribution |
|---|---|---|---|
| Scientific and product workflows | 30% | 27% | 8.1% |
| Intelligence, learning, and knowledge | 15% | 10% | 1.5% |

| Requirement group | Weight | Coverage | Weighted contribution |
|---|---|---|---|
| Extensible capability-node platform | 15% | 25% | 3.8% |
| Enterprise platform | 20% | 4% | 0.8% |
| Role-based UI and customer product | 10% | 10% | 1.0% |
| Cross-domain audit and agent orchestration | 10% | 25% | 2.5% |
| **Total** | **100%** | | **17.7%, rounded to 18%** |

Scientific authority is separate from feature coverage and remains lower. Operational completion, test volume, or signed evidence does not make a module scientifically decision-eligible.

# Enterprise requirement matrix

| Requirement group | Status | Coverage | Existing foundation | Largest gap |
|---|---|---|---|---|
| ProductEnvelope and intake | Missing | 5% | Candidate validation and numeric envelope gates | Source intake, typed requirements, ambiguity review, signature and generation interlock |
| Ingredient/formulation system | Skeleton | 15% | Recipes, cited facts, deterministic arithmetic | Persistent grades/suppliers, formulation CRUD/clone/version locks and live UI |
| Characterization/reconstruction | Skeleton | 10% | Custody-oriented schemas and raw-file hashes | Instrument ingestion, observations, inverse inference, unknown fractions, fingerprints and reconstruction gates |
| Candidate and lab loop | Partial | 40% | Contracts, module outcomes, CRO/report packages, provenance | Unified package, lab issue/return, deviations, disposition and next-batch closure |
| Evolutionary optimization | Foundation | 30% | Generations, score/rank history, lineage and replay | Real formulation operators, eligible fitness, reviewed selection and measured feedback |
| Neuro-symbolic reasoning | Missing | 5% | Deterministic ontology/rule primitives | Probabilistic-symbolic inference, typed hypotheses, ambiguity and search-contract compilation |
| Cross-project learning | Missing | 5% | Rights/provenance/snapshot contracts | Shared knowledge promotion, training exports, model registry and reusable campaign experience |
| Knowledge refinery/library | Skeleton | 10% | Source locks, rights and evidence corpus | Acquisition, normalization, claim review, retrieval service/UI and promotion/rollback |
| Capability-node platform | Foundation | 25% | Static 29-module registry, outcomes, authority inventory | Need matching, node lifecycle records, SDK, packaging, deployment and project lockfiles |
| Enterprise IAM/tenancy | Missing | 2% | Evidence signing identities | Tenants, SSO/MFA/SCIM, RBAC/ABAC, customer admin, isolation and access review |
| APIs/integrations | Missing | 2% | CLI and JSON contracts | Service API, webhooks, SDK and LIMS/ELN/ERP/PLM/QMS/instrument integration |
| Reliability/operations | Skeleton | 10% | Worker pool, artifacts, controller status | Services, queues, SLOs, capacity, RPO/RTO, backup/DR, incidents and support |
| Security/supply chain | Foundation | 20% | Strict parsing, hashes/signatures, read-only controls, gitleaks | Threat model, release/SBOM enforcement, vulnerability program, runtime hardening and incident commitments |
| Deployment profiles | Missing | 2% | Local runtime | Supported SaaS/private/on-prem/air-gap packaging and lifecycle |

| Requirement group | Status | Coverage | Existing foundation | Largest gap |
|---|---|---|---|---|
| Commercial controls | Missing | 0% | None found | Entitlements, quotas, usage, budget, plans and offboarding |
| Role-based web UI | Missing | 8% | Single-run TUI | Six web slices, administration, node registry and audit UI |
| Cross-domain audit service | Foundation | 35% | Hashes, receipts, provenance, replay and archives | Tenant-scoped event service, search/reconstruction UI, retention/redaction/export and DR |
| Agent orchestration | Skeleton | 15% | Paused controller and campaign conventions | Active task graph, registry, tools/credentials, approvals, recovery, UI and evaluation |
| Accessibility/adoption | Missing | 0% | None found | Accessibility, units/time zones, localization, onboarding, training and help |
| Manufacturing release | Missing | 5% | CRO/report and process-QC concepts | Qualified BOM, process window, QC/SPC, change control and release approval |

# Scientific and product workflow gaps

## Intake and signed ProductEnvelope — critical

`CandidateEnvelope` is a strict candidate-message contract, not a partner-intake aggregate. There is no supported document intake, requirement provenance, unit/method normalization, ambiguity register, review UI, signature state, or customer approval gate. Because generation binds to a signed envelope, this is the first missing product boundary.

## Ingredient and formulation management — critical

Candidate recipes, cited ingredient facts, BOM arithmetic, equivalent balance, and envelope checks are useful foundations. Ingredients and formulations remain mostly payloads/reference artifacts, not persistent records. The product needs project applicability, grades, suppliers, costs, restrictions, currentness, immutable referenced versions, cloning/comparison, and an interactive editor.

The existing research BOM is real but is not a supplier-qualified manufacturing BOM. Supplier/grade selection, declarations, domestic/TAA posture, evidence tiers, and final packet completeness remain incomplete.

## Naval characterization and reconstruction — critical

The naval workflow must start with incumbent characterization rather than unconstrained optimization. The system does not implement:

```
sample -> aliquot/fraction -> preparation -> method/instrument -> observation -> component hypothesis -> mass balance/unknown fraction -> fingerprint -> reconstruction
```

Low-level records already bind instruments, calibration hashes, lots, protocols, raw files, endpoints, timestamps, and rights. Missing are supported instrument adapters, analytical QC, method-aware observations, composition hypotheses/ranges, competing explanations, unidentified fractions, fingerprint comparison, baseline formulation/versioning, and review gates. Detailed analysis remains in [MVP-FEATURE-GAP-ANALYSIS.md](MVP-FEATURE-GAP-ANALYSIS.md).

## Candidate compilation and laboratory closure — large

Candidate identity, lineage, evidence envelopes, claim locks, authority, report/CRO packaging, sourcing artifacts, and replay are real. Missing are one durable candidate-package aggregate/UI, complete sourcing/cost/process content, protocol issue/acknowledgement, scheduling, return import, deviation/reconciliation, disposition, and queryable experiment history.

## Evolutionary campaigns — large

The runtime has a mutation–score–rank loop, generations, deterministic seeds, parentage, result history, and replay. The general `EnginePool::mutate` path is explicitly `native-quarantined-no-structure-change-v1`: it changes candidate identity and lineage without changing formulation or structure. Scientific ranking is disabled and no registered module is ranking-eligible.

Required gaps include formulation-aware genomes, authorized mutation/recombination, feasibility repair, retained populations/selection decisions, eligible fitness, empirical feedback, diversity controls, and human review. Existing mechanics are a useful shell, not an active scientific genetic optimizer.

### Manufacturing release — critical but later

Packet and process-QC concepts do not form a release system. Release needs locked composition, supplier-qualified BOM, process window, QC/SPC, batch/change control, sufficiency decisions, signatures, deviations, and external qualification references. Amphora may package authority; it cannot grant NAVSEA/QPL, safety, legal, or procurement approval.

## Capability-node platform assessment

### The baseline is a strong coatings seed

The 29 registered modules cover candidate/evidence contracts, formulation arithmetic and gates, ingredient knowledge, cure/viscosity/solvency/corrosion/process concepts, molecular identity/generation/design concepts, optimizer policy, custody, source rights, replay, red team, sourcing/EHS/IP, CRO packaging, and reporting.

This is a credible initial coatings discovery pack and a better foundation than a generic optimizer without scientific governance. It is not yet a general materials-science pack; batteries, membranes, ceramics, cement, composites, semiconductors, and biological formulations need different representations, methods, tests, failure modes, and process models.

### It is not yet a product node ecosystem

The registry is primarily static native registration plus inventories and file-bound evidence. Missing are:

- `NodeDefinition`, immutable `NodeVersion`, `NodeDeployment`, `NodeInstance`, and `NodeExecution` product records;
- search by question, applicability, input, evidence, authority, cost, health, deployment, and tenant eligibility;
- `CapabilityNeedGraph`, explainable matching, reviewed `CapabilityPlan`, and typed `CapabilityGap`;
- `NodeBuildRequest`, SDK/scaffolder, contract harness, packaging/signing pipeline, and developer portal;
- dependency/schema/unit/representation/version resolution and graph validation;
- customer-private/shared namespaces and deployment management;
- independent operational and scientific lifecycle services;
- complete capability lockfiles across nodes, data, models, knowledge, rules, and tools;
- node owner, reviewer, operator, and project-planner UI.

All 29 modules are `UNVALIDATED`, and none is ranking eligible. Operational readiness cannot substitute for scientific authority.

### One-year minimum node platform

1. A typed node manifest extending the evidence envelope with applicability, schemas, dependencies, resources, rights/security, owner, lifecycle, and validation.
2. A registry service/UI over the baseline nodes.
3. `CapabilityNeed`, `CapabilityPlan`, `CapabilityGap`, and immutable project lockfile records.
4. Deterministic matching with human approval and explicit no-match behavior.
5. One SDK/template path for deterministic nodes and one for adapters/models.
6. Automated contract, known-answer, negative-control, authority, security, and packaging checks.
7. Operational states from development through retirement, separate from scientific authority.
8. Proof by adding at least two project-driven nodes without modifying orchestrator core.

An unrestricted plugin marketplace and arbitrary generated code execution should follow only after sandbox, compatibility, support, and supply-chain controls mature.

# Enterprise platform gaps

## Multi-tenancy and simultaneous customers — critical

Separate processes can technically run concurrently with unique IDs and sufficient resources. That is not enterprise multi-tenancy. No customer/project ownership key, tenant authorization, row/object isolation, database scheduler, quota/admission service, encryption boundary, retention enforcement, tenant audit search, or multi-run operations surface exists.

The required hierarchy is:

```
tenant/customer -> program/project -> experiment/campaign -> run/task/node execution
```

Every artifact, event, query, cache, queue message, node invocation, agent task, export, and metric must respect it. Per-customer deployments may contain early pilots but do not remove the need for an explicit tenancy model.

## IAM — greenfield

No SSO, OIDC/SAML, MFA, SCIM, end-user RBAC/ABAC, customer administration, access certification, session policy, or joiner/mover/leaver workflow was found. Ed25519 evaluator/reviewer identities protect evidence authority; they are not enterprise user IAM.

## Product service, APIs, and integrations — greenfield

Architecture documents propose a React/FastAPI/PostgreSQL application, but it was not found. The supported path is CLI/TUI and local artifacts. Missing are transactional services, persistent domains, durable jobs, object storage, a versioned API, webhooks, SDKs, and reconciliation.

No supported LIMS, ELN, SDMS, ERP, PLM, QMS, procurement, identity-provider, or general instrument integration exists. One year should focus on federation, object storage, generic CSV/file laboratory intake, a CRO exchange, and API/webhooks rather than a broad connector catalog.

## Reliability and continuity — very large

Local worker pools, immutable artifacts, run archives, controller status, and replay are useful. Missing are production services and measurable SLOs, capacity tests, queues, HA stores, backup/restore, RPO/RTO, DR, incident/problem/change management, customer status, service ownership, and support cases. Target values must be selected early because they shape tenancy, queue, database, audit, and deployment architecture.

## Deployment profiles — greenfield

No supported SaaS, private-VPC, on-premises, air-gapped, or hybrid product package was found. Supporting all profiles in one year is unrealistic. Choose one primary and at most one constrained secondary profile; single-tenant hosted/private deployment may be the simplest early containment posture.

## Security and supply chain — foundation only

Strict schemas, hashes, signatures, claim locks, read-only checks, source rights, gitleaks configuration, and adversarial evidence tests are strengths. Enterprise security still needs threat models, service boundaries, IAM/key management, build/release provenance, enforced signing/SBOMs, vulnerability operations, network/egress policy, penetration testing, incident commitments, third-party risk, and deployment hardening.

## Commercial controls — missing

No entitlement, seat, plan, quota, usage, cost allocation, billing export, trial, suspension, renewal, or offboarding model was found. Full invoicing can be deferred for pilots, but entitlements, budgets, hard spend limits, usage attribution, and safe suspension/offboarding cannot.

### Accessibility and adoption — missing

No web application exists to test. Accessibility, supported clients, units/time zones, localization posture, onboarding, training, contextual help, release notes, and privacy-aware analytics remain unimplemented.

## UI gap by slice

| Slice | Existing seed | Missing enterprise outcome |
|---|---|---|
| Operator/DevOps | TUI telemetry and controller status | Fleet/deployment, tenant-safe operations, queues/SLO, incidents/support, DR, secrets and release UI |
| External customer | Report/CRO artifacts | Portal, intake/review, ambiguity/approval, progress, secure exchange, consent and delivery history |
| Internal data scientist | Evidence/model artifacts and CLI | Catalog, exploration, training/evaluation, registry/promotion, retrieval, drift and transfer UI |
| Agent management | Paused controller logs/state | Active registry/topology, tasks, permissions, approvals, costs, evaluation and quarantine UI |
| Customer | Unscoped artifacts | Customer hierarchy, users, rights, portfolio, budgets, consent, residency, timeline and isolation |
| Experiment | Candidate TUI/evidence | Full lifecycle, editor, characterization, custody, protocol, lab returns, results, gates and sign-off |

The TUI is useful as a run/GA prototype, but expanded web-product UI coverage is below 10%.

## Cross-domain auditability gap

This is the strongest conceptual and implementation foundation: canonical JSON, hashes, signed receipts, lineage, claim locks, replay, frozen inputs, refutations, retractions, and supersessions.

Enterprise auditability still requires a shared service with tenant/actor/correlation-bound events, transactional outbox and idempotency, indexed search, timelines/graphs, point-in-time reconstruction, arbitrary diffs, affected-consumer analysis, retention/legal hold/redaction tombstones, break-glass, exports, backup/restore, integrity monitoring, and non-leaking access controls.

The artifact architecture should become the integrity substrate for that service rather than being replaced.

## Agent-orchestration gap

The Claude controller is separate development/research automation, intentionally paused, and not an experiment-time service. The runtime does not call an LLM, neuro-symbolic service, or support agent.

Missing are durable tenant task graphs; agent registry and authority matching; queues, leases, heartbeats, idempotency, retry/compensation/dead-letter recovery; least-privilege tools and credentials; approvals/separation of duties; structured artifact handoff; cost/provider controls; support/incident agents; evaluations and shadow promotion; pause/quarantine/rollback; and complete external-side-effect lineage.

Year one should automate bounded intake, planning, evidence assembly, node scaffolding, status/support, and operator triage. Autonomous scientific promotion, qualification, destructive administration, and self-modifying workflows should remain gated.

## Intelligence, learning, and library gaps

### Neuro-symbolic integration

No active neuro-symbolic engine exists. Ontology/version checks and rules are not joined to a probabilistic hypothesis service. The genetic loop does not invoke neuro-symbolic inference, and the application does not call an LLM.

The minimum year-one proof should be narrow: characterization or envelope observations produce typed hypotheses; deterministic rules/evidence reconcile them; a human approves a search contract; downstream outcomes return for explanation without self-promoting knowledge.

### Cross-project learning

No production cross-project knowledge, model, or optimizer-learning loop exists. The first year should prioritize rights-reviewed reusable knowledge and applicability metadata over pooled model training. Cross-customer training requires enough compatible accepted measurements plus privacy, leakage, rights, validation, and negative-transfer controls.

### Scientific library

The repository can operator-fetch predeclared hash-locked sources and contains extensive references. It has no active discovery/polling service, consolidated claim/corpus product, sparse/dense retrieval service/UI, relevance scheduler, distillation pipeline, promotion/rollback service, or consumption graph. Claude reconnaissance historically discovered metadata under restrictions; it did not dynamically provide experimental evidence and is paused.

## Reusable foundations

Preserve and productize:

- strict candidate parsing, canonical identity, and hashes;
- immutable artifacts, lineage, deterministic seeds, replay, and run history;
- signed receipts and independent evaluator/reviewer concepts;
- typed module outcomes and abstention posture;
- module registry/inventory as the seed node catalog;
- claim locks, authority ceilings, optimizer authorization, and red-team controls;
- rights, source, consumer, snapshot, and activation contracts;
- instrument/calibration/lot/protocol/raw-file evidence bindings;
- CRO, candidate/report, sourceability, EHS, and IP packaging primitives;
- supported deterministic formulation/molecule arithmetic;
- retained attempts, rejects, duplicates, and candidate lineage;
- TUI visualization patterns for future operations and experiment UI.

These reduce integrity and governance risk but do not substitute for persistence, tenancy, services, UI, operations, integrations, or scientific validation.

## Remediation plan

Implementation sequencing, critical path, delivery gates, risks, and deferrals are maintained separately in [ENTERPRISE-REMEDIATION-PLAN.md](ENTERPRISE-REMEDIATION-PLAN.md). This document remains focused on the assessed gap and current acceptance posture.

## Enterprise-pilot acceptance assessment

| Outcome | Current result | Required proof in one year |
|---|---|---|
| Secure simultaneous customers | Not met | Tenant isolation, federation, authorization, quotas, scoped stores/jobs/events and leakage tests |
| Scientist system of record | Not met | Envelope, ingredients, formulations, experiments, samples, measurements, evidence and lab workflow in web UI |

| Outcome | Current result | Required proof in one year |
|---|---|---|
| Extensible node platform | Foundation | Match a project, expose gaps, build two SDK nodes, evaluate/promote/deploy/rollback without core changes |
| Bounded candidate workflow | Partial foundation | Signed envelope, eligible plan, real operators, reviewed fitness/evidence, lab return, selection and replay |
| Naval characterization | Not met | Samples, methods, observations, hypotheses, unknowns, fingerprint, reconstruction and reviewed gate |
| Customer portal | Not met | Intake, ambiguities, approvals, status, evidence-limited results, secure artifacts and consent |
| Enterprise operations | Not met | SLOs, telemetry, queues, backup/restore, RPO/RTO, incidents/support, capacity and degraded modes |
| Enterprise audit | Foundation | Tenant events, search/UI, reconstruction, impact analysis, retention/redaction/export and DR |
| Bounded agents | Not met | Durable tasks, least-privilege tools, approvals, recovery, costs, evaluations and quarantine/rollback |
| Cross-project reuse | Not met | Rights-reviewed shared knowledge promotion, project opt-out and consumption lineage |
| Release/qualification | Not met; external dependency | Evidence-backed release package and external authority; outside core year-one acceptance |

# Final assessment

The repository has already de-risked evidence identity, immutability, provenance, authority ceilings, abstention, replay, and adversarial controls—areas enterprise research systems often postpone. These should remain the product's trust core.

The dominant gap is not simply “more scientific modules.” It is converting a valuable static kernel and module portfolio into a secure, persistent, user-facing, extensible, operable enterprise platform. The baseline node set is sufficient to begin a coatings pilot, but the product must select, validate, compose, build, deploy, support, and retire nodes according to project needs.

The one-year program should be judged by one complete tenant-safe scientific workflow and a proven node-extension path—not module count, evidence-file count, agent activity, or breadth of partially implemented enterprise features.

# Verification notes and limitations

- The assessment compared the enterprise requirements with the workspace manifest, module status/inventory, roadmap/backlog, architecture documents, primary Rust core/engine/CLI/TUI paths, targeted schemas/fixtures, and controller documentation.
- Evidence/reference trees establish artifacts and posture but were not counted as product workflows.
- NOMAD/Crystal and benchkit were excluded unless integrated into the Amphora coatings product.
- No repository-visible frontend, service backend, PostgreSQL product schema, enterprise IAM, or deployment configuration was found.
- This is a feature/architecture assessment, not fresh scientific validation or security certification.
- External labs, supplier qualification, legal/security review, customer acceptance, and naval/manufacturing authority remain external dependencies.